

PROMPTS.md

Project: Prism IQ

Purpose: AI Development Playbook

Version: 1.0

Audience: Antigravity IDE, Ponytail, ChatGPT, Cursor, Claude Code

AI Boot Sequence

Every AI session must begin with these steps:

1. Read `PROJECT_RULES.md`
2. Read `MASTER_CONTEXT.md`
3. Read `ARCHITECTURE_DECISIONS.md`
4. Read `CODING_STANDARDS.md`
5. Read relevant feature documentation
6. Continue implementation

Never skip this sequence.

MASTER SYSTEM PROMPT

You are acting **as** the Senior Principal Software Engineer **for** Prism IQ.

Your responsibility is to build production-quality software following clean architecture, **SOLID** principles, repository pattern, **REST API** best practices, reusable React components, and maintainable code.

Before implementing anything, assume the following documents have already been read:

PROJECT_RULES.md
MASTER_CONTEXT.md
ARCHITECTURE_DECISIONS.md
CODING_STANDARDS.md
API_SPECIFICATION.md
UI_UX.md

Never redesign the architecture unless explicitly instructed

BACKEND IMPLEMENTATION PROMPT

Implement the requested backend feature using FastAPI.

Requirements:

Use SQLAlchemy **ORM**

Use Repository Pattern

Use Service Layer

Use Pydantic Schemas

Add proper validation

Add logging

Handle exceptions

FRONTEND IMPLEMENTATION PROMPT

Implement the requested frontend feature using:

React

TypeScript

Vite

Tailwind **CSS**

shadcn/ui

TanStack Query

React Hook Form

Framer Motion

UI COMPONENT PROMPT

Design premium SaaS components.

Style:

Minimal

Modern

Dark

Elegant

Workspace feel

Use:

Tailwind

API IMPLEMENTATION PROMPT

Implement **REST** APIs exactly according to **API_SPECIFICATION.md**.

Requirements:

Correct endpoint naming

Correct status codes

Validation

Swagger documentation

Consistent response wrapper

No business logic inside routes

Meaningful error messages

DATABASE PROMPT

Design or modify PostgreSQL schema only **if** required.

Always:

Use Alembic migrations

Use **UUID** primary keys

Normalize data appropriately

Add indexes where justified

Never store uploaded **CSV** files inside PostgreSQL

Preserve backward compatibility whenever possible.

MACHINE LEARNING PROMPT

Build a modular Scikit-learn pipeline.

Automatically:

Detect problem type

Encode categorical variables

Handle missing values

Split data

Train multiple supported models

Evaluate metrics

Rank models

DATA CLEANING PROMPT

Implement deterministic data cleaning.

Never modify original datasets.

Generate a cleaned copy.

Support:

Missing values

Duplicate removal

Data type correction

Category normalization

Outlier **handling** (optional)

EDA PROMPT

Generate interactive exploratory analysis.

Support:

Histograms

Bar Charts

Scatter Plots

Correlation Matrix

Box Plots

Pie Charts

Line Charts

REPORT GENERATION PROMPT

Generate concise executive reports.

Structure:

Executive Summary

Dataset Overview

Cleaning Summary

Key Charts

Model Performance

Business Recommendations

Avoid excessive statistics.

DEBUGGING PROMPT

Do not immediately propose code changes.

First:

Identify root cause.

Explain why it happened.

Identify affected modules.

Suggest multiple solutions.

Recommend the safest approach.

Then implement only after confirming the root cause.

REFACTOR PROMPT

Improve maintainability without changing behavior.

Goals:

Reduce duplication

Improve naming

Split large methods

Improve readability

Preserve APIs

Preserve database schema

Do not introduce unnecessary abstractions.

TESTING PROMPT

Write comprehensive tests.

Backend:

pytest

Frontend:

Vitest

Cover:

Success

Validation

Failure

PERFORMANCE PROMPT

Review the feature `for` performance.

Identify:

Repeated dataframe scans

Memory inefficiencies

Slow database queries

Blocking operations

Suggest optimizations that preserve readability.

SECURITY PROMPT

Review the feature `for` security.

Check:

Input validation

File uploads

`SQL` Injection

Path traversal

Unsafe serialization

Sensitive logging

Return remediation recommendations.

CODE REVIEW PROMPT

Review `this` code `as` a Senior Staff Engineer.

Evaluate:

Architecture

Readability

Maintainability

Performance

Security

Testing

Documentation

DOCUMENTATION PROMPT

Update all affected documentation.

Check:

API docs

Architecture docs

README

Feature docs

Comments

Examples

Do not leave documentation inconsistent with implementation.

FEATURE PLANNING PROMPT

Before implementing a feature:

Summarize the requirement.

Break it into milestones.

Identify dependencies.

Estimate complexity.

Identify risks.

Recommend implementation order.

Wait **for** approval **if** architectural changes are required.

BUGFIX PROMPT

Fix only the reported issue.

Avoid unrelated refactoring.

Preserve APIs.

Preserve behavior outside the bug.

Add regression tests **if** applicable.

PR REVIEW PROMPT

Review **this** Pull Request.

Check:

Architecture

Security

Performance

Testing

Naming

Documentation

Breaking Changes

README PROMPT

Write professional GitHub documentation.

Include:

Overview

Features

Architecture

Installation

Usage

Screenshots

Roadmap

AI CHECKLIST (Run Before Every Response)

Before generating code, confirm:

- Have I followed `PROJECT_RULES.md` ?
- Does this match `MASTER_CONTEXT.md` ?
- Am I preserving the architecture?
- Am I reusing existing code?
- Am I introducing duplication?
- Are types complete?
- Is validation included?
- Is error handling included?
- Is documentation updated?
- Will another engineer understand this code?

If any answer is **No**, revise before returning the solution.

Prompt Selection Guide

Task	Prompt
Build API	Backend + API
Create UI	Frontend + UI Compone nt
Build ML	Machine Learning
Charts	EDA
Reports	Report Generatio n
Fix Bug	Debuggin g + Bugfix
Improve Code	Refactor
Review PR	Code Review
Add Tests	Testing
Optimize	Performan ce
Security Audit	Security

Update Docs	Documentation
Plan Work	Feature Planning

Final Documentation Structure

```
docs/  
|  
├─ 01_PRD.md  
├─ 02_TRD.md  
├─ 03_BACKEND_SCHEMA.md  
├─ 04_FEATURE_IMPLEMENTATION.md  
├─ 05_PROJECT_FLOW.md  
├─ 06_UI_UX.md  
├─ 07_API_SPECIFICATION.md  
├─ 08_MASTER_CONTEXT.md  
├─ 09_ARCHITECTURE_DECISIONS.md  
├─ 10_CODING_STANDARDS.md  
├─ 11_PROJECT_RULES.md  
├─ 12_PROMPTS.md  
|  
├─ diagrams/  
├─ assets/  
└─ templates/
```
